



MULTIPLATFORMNÍ KVÍZOVÁ APLIKACE

Maturitní práce

Autor	Jan Housa
Obor	Informační technologie
Vedoucí práce	Mgr. Michal Stehlík
Školní rok	2024/2025
Počet stran	23
Počet slov	4691

Anotace

Tato maturitní práce se zabývá vývojem multiplatformní kvízové aplikace, inspirované platformou Kahoot. Cílem práce je vytvořit interaktivní nástroj, který umožní uživatelům zapojit se do kvízů na různá témata a registrovaným uživatelům nabídne možnost vytvářet vlastní kvízy s různými typy otázek. Implementace probíhá v rámci platformy .NET Blazor, s využitím technologie SignalR pro real-time komunikaci a SQL databáze hostované na Azure.

Summary

This thesis focuses on the development of a multiplatform quiz application inspired by Kahoot. The main goal is to create an interactive tool allowing users to participate in quizzes on various topics, while registered users can create their own quizzes with different question types. The implementation is based on the .NET Blazor framework, using SignalR for real-time communication and an SQL database hosted on Azure.

Čestné prohlášení

Prohlašuji, že jsem předkládanou maturitní práci vypracoval sám a uvedl jsem veškerou použitou literaturu a bibliografické citace.

V Liberci dne 05.06.2025

.....
Jan Housa

Obsah

Úvod.....	1
1 Analýza a návrh aplikace.....	2
1.1 Analýza podobných kvízových platforem.....	2
1.2 Požadavky aplikace	3
1.2.1 Funkční požadavky.....	3
1.2.2 Nefunkční požadavky	3
1.3 Návrh architektury aplikace	4
1.3.1 Základní princip.....	4
1.3.2 Komponenty a propojení	4
1.3.3 Výhody navržené architektury	5
1.3.4 Možné nevýhody a omezení.....	5
1.4 Prototypování a učení se nových technologií.....	6
1.4.1 Experimenty se SignalR.....	6
1.4.2 Vývoj prototypu lobby systému.....	6
1.4.3 Shrnutí.....	7
2 Implementace back-endu.....	8
2.1 Nastavení prostředí a technologií.....	8
2.2 Výhody zvoleného back-end stacku.....	8
2.3 Logika serveru a propojení s databází	9
2.3.1 GameHub	9
2.3.2 Autentizace a autorizace	10
2.3.3 Databázový model.....	11
3 Implementace front-endu	12
3.1 Design a UI/UX aplikace	12
3.2 Navigace v Blazor	12
3.3 Funkční implementace.....	13
3.3.1 SignalRClientService	13
3.4 Nevýhody frontendu	14
3.5 Komplikace při vývoji.....	14
3.5.1 Řešení.....	15
4 Testování.....	17
4.1 Funkční testy.....	17

4.2	Zátěžové testy	17
	Závěr	18
	Seznam zkratek a odborných výrazů	19
	Seznam obrázků	21
	Použité zdroje	22
A.	Seznam příložených souborů	I

Úvod

Digitální technologie představují nedílnou součást moderního vzdělávacího prostředí a zásadně ovlivňují způsoby předávání a přijímání informací. V posledních letech se rozšiřuje využívání inovativních aplikací, které podporují interaktivní výuku, zvyšují motivaci studentů a umožňují aktivnější zapojení všech účastníků vzdělávacího procesu. Tato práce se zaměřuje na návrh a vývoj multiplatformní kvízové aplikace, která je inspirována populární platformou Kahoot, avšak nabízí omezenou funkcionalitu v porovnání s původním řešením. (1) (2)

Vyvíjená aplikace umožňuje interaktivní účast na kvízech v reálném čase a dává registrovaným uživatelům možnost vytvářet vlastní kvízy, čímž se podporuje kreativita a samostatnost při tvorbě obsahu. K realizaci tohoto cíle je využit moderní technologický stack založený na .NET Blazor, který umožňuje vytvářet bohaté a responzivní uživatelské rozhraní bez nutnosti spoléhání se na tradiční webové technologie, jako je JavaScript. Pro zajištění plynulé a okamžité komunikace mezi uživateli byla implementována technologie SignalR, která umožňuje synchronizaci dat mezi serverem a klienty v reálném čase. Je třeba zmínit, že před zahájením projektu nebyly k dispozici předchozí zkušenosti s tvorbou realtime aplikací ani s využitím technologií SignalR a Azure služeb. Tato práce tak sloužila zejména jako prostředek pro osvojování těchto technologií, přičemž byly realizovány četné prototypy, které významně přispěly k pochopení a zdokonalení celého řešení. Je rovněž důležité podotknout, že výsledná aplikace není umístěna na hostingu a proto slouží hlavně jako struktura a koncepční projekt, podle kterého se může někdo inspirovat a dále jej rozvíjet směrem k dokonalosti. (3)

Hlavním cílem této práce je prokázat, že moderní .NET ekosystém, včetně technologií jako Blazor a SignalR, dokáže nabídnout robustní a komplexní řešení pro interaktivní vzdělávání i v případě omezené funkcionality oproti jiným platformám. V rámci práce jsou detailně analyzovány funkční a nefunkční požadavky, navržena odpovídající architektura a prezentovány implementační detaily a výsledky testování vyvinutého řešení, čímž se přispívá k rozvoji moderních vzdělávacích technologií a demonstruje praktická realizace inovativního přístupu ke vzdělávání v digitálním věku.

1 Analýza a návrh aplikace

Tato sekce slouží jako úvodní krok při tvorbě celé aplikace. V ní se čtenář seznámí s kontextem, ve kterém byla aplikace navržena, a s hlavními principy, na kterých stojí její koncept. Cílem této části je představit základní myšlenky a požadavky, jež vedly k vytvoření návrhu architektury. Popisuje se zde, jakým způsobem byly identifikovány klíčové funkce a oblasti, kde je možné dosáhnout optimalizace a inovace. Tato analýza umožňuje pochopit, jakým způsobem se inspirovalo existujícími řešeními a jaké aspekty byly zvoleny pro realizaci interaktivního a uživatelsky přívětivého systému. Celkově tato sekce poskytuje přehled o strategickém plánování a předběžném konceptu, které jsou nezbytné pro vytvoření kvalitního návrhu aplikace.

1.1 Analýza podobných kvízových platforem

Při analýze existujících kvízových platforem, mezi které patří především Kahoot, bylo třeba pečlivě zhodnotit řadu klíčových aspektů, aby bylo možné lépe pochopit, jakým způsobem tyto systémy motivují uživatele a jaké prvky přispívají k jejich úspěšnosti. Aplikace Kahoot se proslavila zejména díky své schopnosti podpořit soutěživost a okamžitou zpětnou vazbu, což výrazně zvyšuje zapojení účastníků během výuky. Mezi její hlavní vlastnosti patří například jednoduchý přístup k účasti prostřednictvím PIN kódu, který umožňuje rychlé a bezproblémové připojení, okamžitý výpočet bodů, který dodává soutěžnímu prostředí na dynamice, a interaktivní tabule, jež průběžně zobrazuje aktuální pořadí účastníků. Kromě toho podporuje Kahoot i různé typy otázek, čímž zajišťuje pestrost a flexibilitu výukového procesu. (2)

Analýza platformy Kahoot tak odhalila nejen silné stránky tohoto systému, ale také oblasti, kde je možné nabídnout zjednodušený proces nebo rozšířit funkčnost s ohledem na specifické potřeby moderního vzdělávání. Zvláštní pozornost byla věnována možnosti vytvoření aplikace, která není závislá na JavaScriptu, což představuje významnou výhodu pro vývojáře se specializací na C# a .NET. Nicméně absence JavaScriptu se ukázala jako omezující faktor u některých funkcí, jelikož samotný Blazor nedosahuje efektivity tradičních JavaScriptových řešení v určitých ohledech. Z tohoto důvodu musela být aplikace omezena na sadu funkcí, které jsou Blazorem plně efektivně podporovány, což vedlo k vynechání některých prvků, jež by jinak mohly zvýšit interaktivitu a komplexnost systému. (2) (4)

Celkově analýza aplikace Kahoot poskytla cenné poznatky, které vedly k formulaci požadavků na nově vyvíjenou kvízovou aplikaci. Díky tomu bylo možné identifikovat klíčové prvky, jež by měly být zachovány pro zajištění interaktivity a motivace, a zároveň odhalit oblasti, kde lze dosáhnout zjednodušení a optimalizace celého systému s ohledem na technologická omezení. Tento přístup podtrhuje význam důkladné

předběžné analýzy při vývoji nových vzdělávacích technologií, které mají za cíl nejen replikovat úspěšné vlastnosti stávajících řešení, ale také přinášet inovace odpovídající aktuálním potřebám uživatelů i vývojářů. (2)

1.2 Požadavky aplikace

V této sekci jsou definovány klíčové požadavky, které musí aplikace splňovat, aby byla funkční, bezpečná a uživatelsky přívětivá. Požadavky jsou rozděleny do dvou hlavních kategorií:

1.2.1 Funkční požadavky

Registrace a přihlášení jsou základní funkcí aplikace. Uživatelé si mohou vytvořit účet, což jim umožňuje přístup k rozšířeným funkcím, jako je tvorba vlastních kvízů a správa jejich obsahu. Pro uživatele, kteří nechtějí nebo nemohou účet vytvořit, je připraven režim hosta, díky němuž se mohou jednoduše zapojit do kvízů bez nutnosti registrace.

Registrovaní uživatelé mají možnost sestavovat kvízy s různými typy otázek, jako jsou otázky s odpověďmi ve formě Ano/Ne nebo s více možnostmi. Při tvorbě kvízu mohou doplňovat otázky o obrázky a definovat správné odpovědi, čímž zvyšují atraktivitu a interaktivitu obsahu.

Spouštění kvízů probíhá prostřednictvím herních lobby, které zakládá vlastní kvízu. Do těchto lobby se ostatní hráči připojují pomocí unikátního PIN kódu. Během hry se automaticky počítají body za rychlé a správné odpovědi, přičemž výsledky se okamžitě promítají do dynamického score boardu. Tento systém bodování přináší soutěživý prvek, který motivuje hráče ke zlepšování svých výkonů.

1.2.2 Nefunkční požadavky

Aplikace musí kromě funkčních požadavků splňovat i řadu nefunkčních požadavků, které jsou zásadní pro její spolehlivost a pohodlné používání.

Jedním z klíčových nefunkčních požadavků je škálovatelnost systému. Aplikace by měla být schopna efektivně zpracovávat více současně připojených uživatelů, což je zásadní pro zajištění plynulého a bezproblémového provozu v různých scénářích využití. Přestože aktuální implementace využívá Azure předplatné s omezením na 20 současných připojení, návrh systému je navržen s ohledem na možnost budoucího rozšíření a zvýšení počtu uživatelů.

Dalším požadavkem je nízká latence. Díky implementaci technologie SignalR se minimalizuje zpoždění při přenosu dat mezi serverem a klientem, což je nezbytné pro udržení plynulého herního zážitku a okamžité aktualizace herního stavu. (3)

Bezpečnost je rovněž prioritou. Aplikace využívá moderní autentizační mechanismy, jako je JWT, a dodržuje principy ASP.NET Core Authorization, což zajišťuje ochranu citlivých dat a bezpečné připojení uživatelů. (5)

Uživatelské rozhraní musí být plně responzivní. Tato vlastnost zaručuje, že aplikace bude bezproblémově přístupná na desktopu, tabletu i mobilním telefonu, což zvyšuje celkový komfort uživatelů.

1.3 Návrh architektury aplikace

Tato sekce představuje podrobný návrh struktury aplikace, který vychází z principu oddělení odpovědností do několika vrstev.

1.3.1 Základní princip

Architektura aplikace je rozdělena do tří hlavních vrstev: uživatelské rozhraní, serverová logika a databázová vrstva. Tento přístup umožňuje jasné oddělení odpovědností, což značně usnadňuje údržbu a případné budoucí rozšiřování systému.

Na straně front-endu využívá aplikace technologii Blazor ve dvou variantách – Server-Side a WebAssembly. Tento přístup umožňuje vytvářet moderní, interaktivní a responzivní uživatelské rozhraní bez nutnosti psaní rozsáhlého množství JavaScriptu, což vede k zjednodušení vývojového procesu a usnadňuje správu kódu. (6)

Back-end aplikace je postaven na ASP.NET Core, který zajišťuje zpracování REST API, autentizaci a real-time komunikaci pomocí SignalR. Tato vrstva představuje jádro logiky aplikace a stará se o bezpečný přístup k datům, přičemž efektivně řídí komunikaci mezi klienty a serverem. Robustní design této vrstvy zaručuje, že všechny operace probíhají spolehlivě s ohledem na výkon a stabilitu systému. (3)

Databázová vrstva využívá SQL Server hostovaný v Azure, který slouží jako úložiště pro veškerá data – informace o kvízech, otázkách, odpovědích a uživateli. Použití Entity Framework Core usnadňuje manipulaci s databázovými entitami a umožňuje flexibilní správu datového modelu, čímž se zajišťuje robustnost a škálovatelnost systému, což je klíčové pro aplikaci s vysokým počtem uživatelů. (7)

1.3.2 Komponenty a propojení

Navržená architektura je založena na modulárním přístupu, kdy jsou jednotlivé komponenty systematicky oddělené, avšak navzájem úzce spolupracují prostřednictvím jasně definovaných rozhraní. Tento přístup umožňuje snadnější údržbu, rozšiřování a případnou výměnu jednotlivých částí systému bez narušení celkové funkčnosti aplikace. (8)

Technologie Blazor Server a SignalR společně umožňují obousměrnou komunikaci mezi klientem a serverem. Každá změna na straně klienta, například výběr odpovědi nebo jiná interakce s uživatelským rozhraním, je okamžitě zaznamenána a odeslána na server, který danou akci zpracuje. Výsledky jsou poté v reálném čase distribuovány všem připojeným uživatelům prostřednictvím SignalR, což zajišťuje, že všichni hráči mají aktuální informace a mohou interagovat se systémem synchronizovaně. Tato metoda přispívá k výraznému zvýšení interaktivity a dynamiky hry.

Databázová vrstva využívá Entity Framework Core, který umožňuje automatické mapování databázových entit na objekty v aplikaci. Tento přístup značně usnadňuje správu dat, jejich validaci a provádění migrací mezi různými verzemi aplikace. Díky tomu je možné efektivně pracovat s databázovým modelem, zajišťovat konzistenci dat a snadno implementovat potřebné změny během vývoje či rozšiřování aplikace. Modulární struktura databázové vrstvy umožňuje rychlé nasazení aktualizací a zvyšuje flexibilitu celého systému. (7)

Celkově tento modulární přístup podporuje agilní vývoj, kdy lze jednotlivé komponenty nezávisle testovat, ladit a optimalizovat, což přispívá k vyšší robustnosti aplikace a zaručuje, že budoucí rozšíření nebo změny lze implementovat s minimálním dopadem na ostatní části systému.

1.3.3 Výhody navržené architektury

Modularita: Díky jasnému oddělení jednotlivých vrstev je systém snadno udržitelný a lze jej dále efektivně rozšiřovat. Každá vrstva je navíc možné nezávisle testovat a optimalizovat, což přispívá k celkové stabilitě a flexibilitě aplikace. (8)

Jasně oddělení logiky: Front-end (uživatelské rozhraní), back-end (obchodní logika) a databázová vrstva (data) jsou navrženy jako samostatné komponenty, čímž se zabráňuje vzniku monolitického kódu. Toto rozdělení umožňuje efektivnější správu a poskytuje lepší přehled o celkové architektuře systému.

Srozumitelný kód: Využití ASP.NET Core a Entity Framework Core snižuje křivku učení pro vývojáře se zkušenostmi s .NET stackem. Díky tomu je kód nejen přehledný, ale také snadno pochopitelný a udržitelný, což usnadňuje budoucí rozvoj a případné úpravy aplikace.

1.3.4 Možné nevýhody a omezení

Závislost na .NET stacku: Aplikace je úzce svázána s technologiemi .NET a C#, což může omezit možnosti rozšíření nebo migrace do jiných technologických prostředí. (1)

Potenciální náklady na Azure: U větších projektů s vysokým počtem uživatelů mohou náklady spojené s hostováním a správou databáze v Azure růst, což je nutné pečlivě plánovat a sledovat. (9)

Potřeba stabilní konektivity: Real-time komunikace prostřednictvím SignalR vyžaduje stabilní a rychlé internetové připojení na straně klienta i serveru, což nemusí být vždy zaručeno v každém prostředí.

1.4 Prototypování a učení se nových technologií

Před zahájením vývoje hlavních částí aplikace – backendu a frontendu – bylo nezbytné realizovat několik prototypů, které posloužily k ověření funkčnosti vybraných technologií a zároveň k získání praktických zkušeností s jejich implementací. Tento proces prototypování byl klíčový pro pochopení možností a omezení jednotlivých nástrojů a frameworků, zejména pak technologie SignalR, která sehrála zásadní roli v zajištění real-time komunikace mezi klientem a serverem.

1.4.1 Experimenty se SignalR

První fáze experimentování se SignalR začala implementací jednoduché chatovací aplikace. Tento prototyp umožnil týmu seznámit se se základními principy vytváření a správy připojení, odesílání a přijímání zpráv a správou skupin uživatelů. Díky této zjednodušené verzi bylo možné rychle ověřit, jak SignalR funguje v praxi, a zjistit, jak snadno se dá integrovat do .NET ekosystému. Úspěšné dokončení tohoto prototypu přineslo nejen důležité poznatky o asynchronní komunikaci, ale také otevřelo cestu k rozšíření funkcionality v dalších fázích vývoje. (3)

1.4.2 Vývoj prototypu lobby systému

Na základě získaných zkušeností z chatovací aplikace byl dále vyvinut prototyp lobby systému. Tento systém měl za úkol umožnit správu herních místností, do kterých se uživatelé mohli připojovat pomocí unikátního PIN kódu. V rámci tohoto prototypu bylo třeba řešit několik zásadních aspektů:

Správa připojení: Bylo nutné zajistit, aby se při vstupu do lobby odstranila duplicitní připojení a aby byl první připojený uživatel automaticky označen jako vlastník místnosti.

Synchronizace dat: Protokolování akcí uživatelů, jako je připojování a odpojování, odesílání zpráv či aktualizace seznamu hráčů, mělo probíhat v reálném čase. Tato synchronizace byla klíčová pro zajištění konzistence stavu herní místnosti pro všechny připojené klienty.

Vývoj prototypu lobby systému tak poskytl cenné poznatky nejen o technické implementaci, ale i o návrhu uživatelského rozhraní a celkové architektuře aplikace. Zkušenosti získané při tvorbě tohoto prototypu významně přispěly k rozhodnutím, která byla učiněna během návrhu finální verze aplikace.

1.4.3 Shrnutí

Prototypování a experimentování s novými technologiemi, jako je SignalR, hrálo klíčovou roli v celém vývojovém procesu aplikace. Implementace jednoduché chatovací aplikace umožnila základní seznámení se s principy real-time komunikace, zatímco následný vývoj prototypu lobby systému poskytl hlubší vhled do komplexnější správy herních místností a synchronizace dat mezi více uživateli. Tyto zkušenosti byly zásadní pro finální návrh a implementaci aplikace, která nyní využívá osvědčené postupy a moderní technologie pro zajištění interaktivního a spolehlivého herního prostředí. (1)

2 Implementace back-endu

V této části práce je popsán proces vývoje serverové logiky aplikace. Sekce obsahuje popis nastavení vývojového prostředí, výběru technologií a metod implementace REST API, real-time komunikace prostřednictvím SignalR a bezpečné integrace s SQL databází hostovanou na Azure. Důraz je kladen na bezpečnost, validaci dat a robustnost řešení.

2.1 Nastavení prostředí a technologií

Pro vývoj back-endu byla zvolena nejnovější verze .NET 8, která přináší řadu vylepšení a optimalizací oproti předchozím verzím. Vývoj probíhal v prostředí Visual Studio 2022, což umožnilo využití pokročilých funkcí pro debugování a správu projektů.

Klíčové technologie zahrnují ASP.NET Core, který zajišťuje běh webového serveru, tvorbu REST API a hostování SignalR hubu. Jeho modularita umožňuje rychlý vývoj a snadnou integraci s dalšími komponentami, což výrazně usnadňuje tvorbu robustního back-endu. Entity Framework Core je ORM nástroj, který mapuje data z databáze na objekty v kódu. Díky tomu se výrazně zjednodušuje správa entit a provádění migrací, což umožňuje efektivní a flexibilní práci s databází. (4) (7)

SQL Azure představuje cloudové databázové řešení, jež nabízí vysokou dostupnost, škálovatelnost a bezpečnost. Díky integrovaným nástrojům pro monitoring a optimalizaci výkonu je možné snadno sledovat a upravovat chování databáze podle aktuálních potřeb. SignalR umožňuje real-time komunikaci mezi klientem a serverem, což je klíčové pro interaktivní kvízy, kde je důležitá minimální latence a plynulost přenosu informací. Pro autentizaci a autorizaci uživatelů se využívá JWT (JSON Web Token), který zajišťuje bezpečnou komunikaci a přístup k citlivým údajům. Tento mechanismus posiluje ochranu dat a zaručuje, že přístup k API mají pouze oprávnění uživatelé. (10)

K verzování kódu byl využit Git a vzdálený repozitář na GitHubu. Tím je zajištěn přehled o všech změnách a možnost snadného návratu k předchozím verzím. (11)

2.2 Výhody zvoleného back-end stacku

Použití ASP.NET Core a EF Core přináší několik klíčových výhod. První z nich je rychlý vývoj – moderní frameworky umožňují tvorbu a nasazení aplikace v kratším čase, což výrazně zkracuje dobu potřebnou k uvedení projektu do provozu. Další významnou výhodou je bezpečnost; integrované autentizační mechanismy, jako je například JWT, poskytují vysokou úroveň ochrany dat a minimalizují riziko zneužití citlivých informací. Škálovatelnost představuje další benefit, neboť lze snadno přidávat další servery a moduly dle potřeby rostoucího počtu uživatelů. Kromě toho podpořená rozsáhlou

dokumentací a aktivní komunitou vývojářů usnadňuje řešení případných problémů, což celkově zvyšuje efektivitu vývojového procesu. (1)

2.3 Logika serveru a propojení s databází

Back-end aplikace je navržen jako dvouvrstvá architektura, která jasně odděluje logiku zpracování požadavků od práce s databází. Tento přístup přináší několik významných výhod, které se pozitivně projevují na celkové stabilitě a bezpečnosti systému.

První výhodou je modulární rozdělení odpovědností. Kontrolery a SignalR huby se starají o příjem a zpracování požadavků, zatímco databázová vrstva je zodpovědná za správu a ukládání dat. Toto oddělení umožňuje, aby každá vrstva mohla být nezávisle testována a optimalizována, což výrazně usnadňuje údržbu a budoucí rozšiřování aplikace.

Dalším klíčovým aspektem je bezpečné zpracování dat. Všechna data přicházející od uživatelů jsou důkladně validována před tím, než jsou uložena do databáze s využitím ORM. Tento postup minimalizuje riziko SQL injection a dalších bezpečnostních hrozeb, čímž se zajišťuje integrita a ochrana citlivých informací.

Nakonec, díky SignalR hubu je zajištěna real-time komunikace mezi serverem a klienty. Každá změna v systému, například připojení nového hráče, je okamžitě sdělena všem připojeným klientům. Tato okamžitá synchronizace podporuje dynamickou interaktivitu aplikace a přispívá k hladkému chodu reálného času, což je zásadní pro interaktivní prostředí, jako jsou kvízové hry.

2.3.1 GameHub

GameHub je klíčová komponenta aplikace zajišťující real-time komunikaci mezi serverem a klienty během kvízových her pomocí SignalR. Řídí připojování a odpojování uživatelů, správu herních místností (lobby), odesílání otázek, přijímání odpovědí a aktualizaci skóre hráčů, čímž umožňuje okamžitou interakci a sledování průběhu kvízu.

Při vstupu do lobby se hráč přihlásí metodou `JoinLobby`, která ho přidá do skupiny definované unikátním PIN kódem. Během tohoto procesu jsou odstraněna duplicitní připojení a první hráč se stává vlastníkem lobby, což určuje hierarchii – pouze vlastník může spustit hru či odesílat otázky.

Metoda `StartGame` spouští kvíz, přičemž kontroluje, zda akci provádí vlastník. Jakmile hra začíná, hub inicializuje datové struktury pro zaznamenávání odpovědí a pomocí SignalR posílá zprávy klientům, aby byli přesměrováni na herní stránku, což synchronizuje stav aplikace pro všechny zúčastněné.

Odesílání otázek se realizuje metodou `SendQuestion`, která umožňuje vlastníkovu lobby poslat otázku spolu s obrázkem, seznamem odpovědí a definicí správné odpovědi. Po

odeslání otázky je spuštěn časový odpočet, během kterého mohou hráči odpovídat. Po vypršení času GameHub vyhodnotí odpovědi, aktualizuje skóre na základě správných odpovědí a rozesílá výsledky zpět všem klientům, čímž zajišťuje transparentní a dynamickou interakci.

V případě odpojování hráče metoda `OnDisconnectedAsync` odstraní uživatele ze seznamů a, pokud odchází vlastník, dojde k jeho nahrazení. Seznam hráčů a jejich avatarů se pravidelně aktualizuje metodou `UpdateLobbyUsers` a posílá všem členům lobby. Navíc metoda `SendMessage` umožňuje okamžité zasílání textových zpráv mezi účastníky.

Celkově GameHub zásadně přispívá ke správě interakce a synchronizaci událostí mezi hráči. Díky SignalR je zajištěn plynulý přenos informací, což je nezbytné pro dynamické prostředí interaktivních kvízů a přispívá k robustnosti a uživatelské přívětivosti aplikace.

2.3.2 Autentizace a autorizace

Autentizace a autorizace v back-endu využívají JWT (JSON Web Token) pro bezpečný přístup k citlivým operacím a ochranu dat. Tento mechanismus je integrován přímo do API a realizován pomocí controllerů, knihovny pro generování tokenů a vlastního middleware, který validuje příchozí požadavky. (12)

Při registraci (endpoint „register“) se nejprve ověřuje, zda zvolené uživatelské jméno není již obsazeno. Pokud je registrace úspěšná, heslo se zabezpečí hashováním pomocí HMACSHA512 – uloží se pouze hash a salt, což výrazně zvyšuje bezpečnost, neboť heslo není uloženo v čitelné podobě.

Při přihlášení (endpoint „login“) se ověřuje identita uživatele opět pomocí HMACSHA512. Po úspěšném ověření hesla systém vygeneruje JWT token, který obsahuje informace o uživateli – uživatelské jméno, ID a roli – a je podepsán symetrickým klíčem z konfigurace. Token má definovanou expiraci (např. jednu hodinu), což omezuje dobu jeho platnosti.

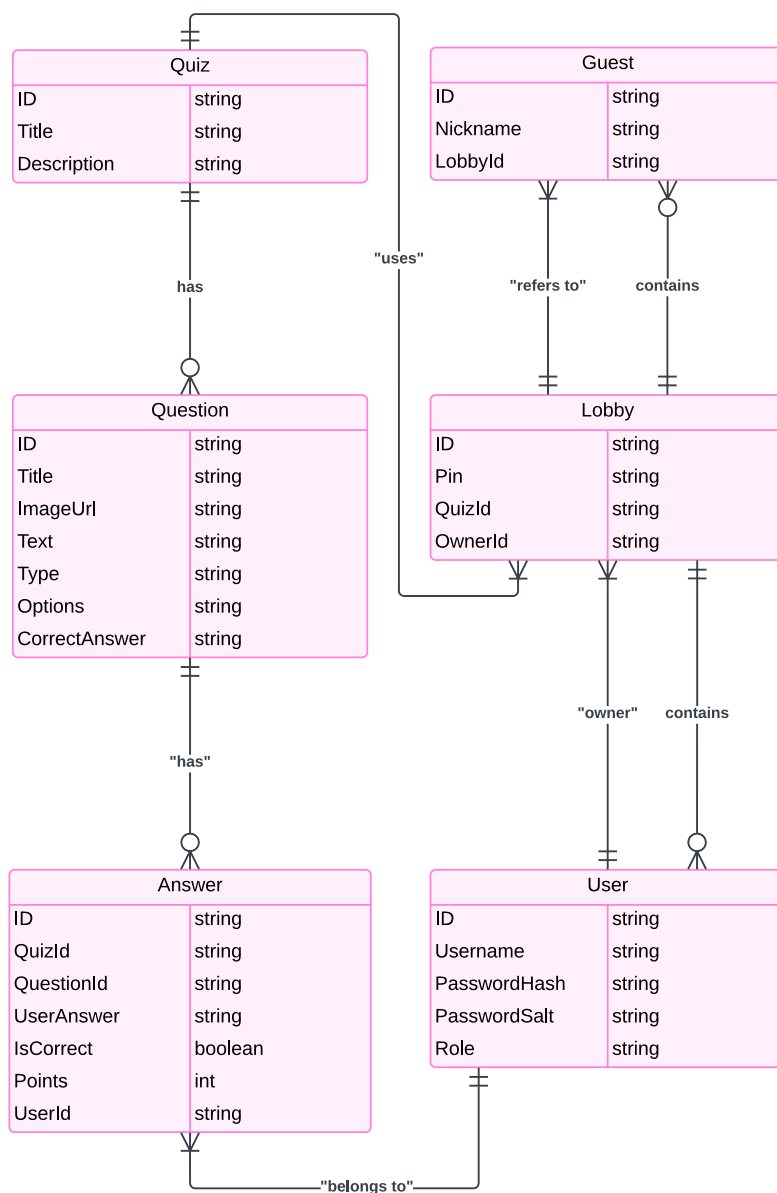
Vlastní middleware pro validaci tokenu čte JWT z hlavičky každého požadavku, dešifruje jej pomocí knihovny Jose a získává z něj claimy (například jméno, ID a roli). Po úspěšném ověření se tyto informace vloží do objektu `HttpContext.User`, což umožňuje dalším komponentám přístup k identitě a oprávněním uživatele. V případě neplatného či chybějícího tokenu middleware zastaví zpracování požadavku a vrátí odpověď s kódem 401 Unauthorized. (13)

Tento centralizovaný přístup k autentizaci a autorizaci zaručuje, že pouze oprávnění uživatelé mají přístup k chráněným zdrojům a operacím. Kombinace bezpečného ukládání hesel, robustního generování tokenů a efektivní validace požadavků významně přispívá k celkové bezpečnosti a spolehlivosti systému.

2.3.3 Databázový model

Databázový model aplikace definuje klíčové entity, které reprezentují reálné objekty v systému, jako jsou kvízy, otázky, odpovědi, uživatelé, hosté a herní místnosti (lobby). Každá entita zahrnuje atributy potřebné pro správu a uchovávání příslušných informací. Tyto entity jsou mapovány na databázové tabulky pomocí Entity Framework Core, což umožňuje efektivní správu vztahů mezi objekty a zajišťuje konzistenci dat.

Tento přístup usnadňuje provádění operací CRUD, validaci dat, migraci a další údržbu databázového modelu. Pro lepší vizualizaci struktury a vztahů mezi jednotlivými entitami je databázový model dále ilustrován pomocí diagramu, který poskytuje přehled o celkové architektuře datového úložiště. (1)



Databázový Diagram 1

3 Implementace front-endu

Tato sekce popisuje vývoj uživatelského rozhraní aplikace. Zaměřuje se na návrh designu a UX, implementaci navigace pomocí Blazor a integraci s back-endovými službami. Je zde zdůrazněna dynamická interaktivita, responzivita napříč zařízeními a řešení specifických problémů spojených s generováním dynamických komponent.

3.1 Design a UI/UX aplikace

Uživatelské rozhraní je postaveno převážně na Bootstrapu, což zajišťuje konzistenci, přehlednost a plnou responzivitu napříč zařízeními – od desktopů po tablety a mobily. Díky „mobile-first“ přístupu se rozhraní automaticky přizpůsobuje různým velikostem displejů a optimalizuje načítání, což výrazně zkracuje dobu čekání a poskytuje kvalitní uživatelský zážitek. V případech, kdy základní styly Bootstrapu nevyhovovaly, byly použity dodatečné CSS úpravy pro dosažení požadovaného vizuálního dojmu. (14)

Dynamická interaktivita je zajištěna technologií Blazor, která umožňuje okamžité aktualizace obsahu bez nutnosti načítání celé stránky. Tento přístup přispívá k plynulému ovládání a rychlému zobrazení změn, což výrazně podporuje efektivní práci se systémem a zvyšuje uživatelskou spokojenost. (15)

Jednoduchost a přehlednost rozhraní byla dále podpořena sjednoceným vzhledem aplikace, kde jednotné barvy, fonty a rozložení prvků zaručují intuitivní navigaci. Přehledné formuláře pro přihlášení, registraci či zadání PIN kódu a jasně definované ikony a tlačítka pro základní funkce napomáhají rychlé orientaci a efektivní komunikaci se systémem.

3.2 Navigace v Blazor

Navigace v aplikaci je řízena pomocí direktiv `@page` v `.razor` souborech, což umožňuje plynulý přechod mezi jednotlivými stránkami. Uživatel může například snadno přecházet ze stránky pro zadání PINu na přehled dostupných kvízů, dále do detailu konkrétního kvízu a nakonec do herního prostředí. Tento způsob navigace zajišťuje, že se načítají pouze potřebné části obsahu, což eliminuje potřebu reloadu celé stránky a výrazně zrychluje uživatelskou interakci. (16)

Mezi hlavní výhody tohoto přístupu patří okamžité načítání obsahu a dynamická aktualizace stránky, což zvyšuje komfort a plynulost práce s aplikací. Například při přechodu z hlavní stránky do detailu kvízu se uživateli okamžitě zobrazí informace o vybraném kvízu bez zbytečného čekání na opětovné načtení celé stránky – v tradičním přístupu by reload mohl způsobit dočasnou ztrátu kontextu nebo zpomalení interakce.

Silná integrace s .NET ekosystémem dále přispívá ke konzistenci logiky a snadné správě kódu. Na druhou stranu, tento způsob navigace může mít omezenou flexibilitu ve

srovnání s některými tradičními JavaScriptovými frameworky. U velmi specifických interakcí nebo rozsáhlé klientské logiky, kde by nativní JavaScript nebo knihovny jako React či Angular mohly nabídnout jemnější kontrolu a lepší výkon, může být Blazor méně vhodný. Celkově však přístup založený na direktivách `@page` poskytuje vyvážené řešení pro většinu scénářů a usnadňuje vývoj jednotného a responzivního uživatelského prostředí. (1)

3.3 Funkční implementace

Funkčnost front-endu aplikace je postavena na technologii Blazor, která integruje různé aspekty interakce s uživatelem a komunikace se serverem. Mezi klíčové funkce patří autentizace, komunikace se serverem pomocí různých komunikačních kanálů a dynamické zobrazení stavu hry.

Pro autentizaci jsou v aplikaci využívány zabezpečené tokeny, které jsou generovány na straně serveru a poté uloženy v LocalStorage prohlížeče. Tyto tokeny se následně připojují k hlavičkám každého HTTP požadavku, což umožňuje přístup k autorizovaným akcím, jako je vytváření nových kvízů nebo správa herních místností (lobby). Tento přístup zajišťuje, že pouze ověření uživatelé mají přístup k citlivým funkcím a operacím, a tím posiluje celkovou bezpečnost systému.

Komunikace se serverem je řešena dvěma způsoby. Pro standardní požadavky, jako je vytvoření nového kvízu, načtení otázek nebo registrace uživatele, se používá HttpClient. Tento nástroj umožňuje odesílání REST API požadavků a následné zpracování odpovědí, což zajišťuje efektivní komunikaci mezi klientem a serverem. Na druhé straně, pro real-time interakci se využívá technologie SignalR. SignalR umožňuje okamžitý přenos událostí, jako je vysílání nové otázky, aktualizace pořadí hráčů nebo změny ve stavu hry, a tím zaručuje, že uživatelé dostávají aktuální informace bez zpoždění.

Díky těmto technologiím je možné v reálném čase zobrazovat aktuální stav hry přímo na stránce herní lobby. Uživatelé zde vidí seznam aktuálně připojených hráčů, jejich aktuální skóre a aktuálně aktivní otázku. Tento dynamický přístup výrazně zlepšuje herní zážitek, jelikož umožňuje okamžitou zpětnou vazbu a plynulou interakci mezi účastníky kvízu. Celkově funkční implementace zajišťuje bezpečnou, efektivní a dynamickou komunikaci, která podporuje jak standardní, tak real-time scénáře použití. (1)

3.3.1 SignalRClientService

SignalRClientService je klíčová služba na front-endu, která zajišťuje real-time komunikaci s back-endem pomocí technologie SignalR. Tato služba využívá knihovnu Microsoft.AspNetCore.SignalR.Client k vytvoření a správě HubConnection, což umožňuje okamžité odesílání a přijímání zpráv mezi klientem a serverem.

SignalRClientService zajišťuje, že před odesláním jakéhokoliv požadavku je spojení aktivní, a pokud není, vyvolá výjimku. Prostřednictvím metody StartConnection se naváže spojení na specifikovanou URL a automaticky se provádí opětovné připojení v případě výpadku. Dále služba umožňuje registrovat callbacky pro různé události, jako je příjem zpráv, aktualizace času odpočtu, zobrazení otázek, nebo aktualizace skóre hráčů, čímž zajišťuje dynamickou interakci v reálném čase.

Kromě real-time funkcí poskytuje SignalRClientService také metody pro volání specifických akcí na serveru, jako je připojení do lobby, odeslání zprávy, spuštění hry, odeslání odpovědi, nebo zaslání otázek. Tímto způsobem integruje funkce REST API a SignalR, což umožňuje plynulou a efektivní komunikaci mezi front-endem a back-endem.

3.4 Nevýhody frontendu

Navzdory řadě výhod má implementace frontendu s využitím Blazoru a Bootstrapu i některé nevýhody. Hlavní omezení spočívá v menší flexibilitě a možnosti implementace pokročilejších funkcí ve srovnání s nativními JavaScriptovými frameworky, jako jsou React nebo Angular. Některé specifické interakce a rozsáhlá klientská logika mohou být tak v Blazoru méně efektivní. Dalším faktorem je využití Bootstrapu, který, ačkoliv urychluje vývoj a zajišťuje konzistentní vzhled, často vede k podobnému vizuálnímu stylu, jenž se může opakovat napříč různými projekty. Tyto aspekty mohou omezit možnost vytvoření unikátního designu a diferenciaci od ostatních aplikací. (1)

(14)

3.5 Komplikace při vývoji

Během vývoje frontendu došlo k potížím při dynamickém generování vstupních polí pro možnosti otázek v kvízu. Konkrétně se problém projevil tak, že při zadávání textu do těchto polí se hodnoty okamžitě ztrácely a text se neustále přepisoval. Příčinou bylo, že inputy byly vytvářeny v cyklu bez dostatečně jedinečných identifikátorů, což způsobovalo, že Blazor při každém renderování nedokázal správně rozlišit jednotlivé komponenty a následně je nesprávně „recykloval“.

Hodnoty v dynamicky generovaných inputech byly při každém renderu resetovány. To bylo způsobeno tím, že Blazor neměl možnost udržet stav jednotlivých komponent, což vedlo ke ztrátě uživatelského vstupu. Například původní kód generoval inputy bez jedinečných klíčů:

```
@for (int j = 0; j < newQuestion.Options.Count; j++)
{
    <div class="input-group mb-2">
        <InputText class="form-control"
            @bind-Value="newQuestion.Options[j]"
            placeholder="@($"Option {j + 1}")" />
    </div>
}
```

Interní model přecházel z používání prostých řetězců na strukturovaný model – třídu `OptionModel` s vlastnostmi `Id` a `Text`. Tato změna způsobila problém, protože API očekávalo seznam řetězců, nikoli objektů, což vedlo k validační chybě při odesílání quizu.

3.5.1 Řešení

Do smyčky generující inputy byl přidán atribut `@key`, který umožňuje Blazoru správně sledovat jednotlivé komponenty. Upravený kód s využitím jedinečného identifikátoru vypadá následovně:

```
@for (int j = 0; j < newQuestion.Options.Count; j++)
{
    var option = newQuestion.Options[j];
    <div class="input-group mb-2" @key="option.Id">
        <InputText class="form-control"
            @bind-Value="option.Text"
            placeholder="@($"Option {j + 1}")" />
    </div>
}
```

Namísto seznamu řetězců byly možnosti reprezentovány objekty třídy `OptionModel`. Definice této třídy vypadá například takto:

```
public class OptionModel
{
    public int Id { get; set; }
    public string Text { get; set; }
}
```

Protože API očekávalo seznam řetězců, před odesláním quizu došlo k transformaci interních modelů na strukturu vyhovující API. V metodě `HandleValidSubmit` byl vytvořen anonymní objekt, který extrahuje pouze hodnoty `Text` z každé možnosti:

```
options = q.Options.Select(o => o.Text).ToList()
```

Tento úsek kódu ukazuje, jak se pomocí LINQ převádí kolekce objektů `OptionModel` na seznam řetězců, který API očekává. (1)

4 Testování

Sekce shrnuje provedené testovací postupy, včetně funkčních a zátěžových testů. Popisuje metody ověření správného fungování klíčových funkcí aplikace, měření odolnosti systému při reálné uživatelské zátěži a vyhodnocení výsledků, které potvrzují spolehlivost a efektivitu navrženého řešení.

4.1 Funkční testy

Funkční testy ověřovaly, zda všechny klíčové funkce aplikace pracují podle očekávání. Testovány byly scénáře jako registrace a přihlášení uživatelů, vytváření a správa kvízů, správné fungování herních lobby a real-time komunikace prostřednictvím SignalR. Výsledky testů potvrdily, že uživatelské rozhraní reaguje správně na akce, všechny požadované funkce jsou implementovány a systém správně zpracovává vstupy od uživatelů.

4.2 Zátěžové testy

Zátěžové testy se zaměřily na odolnost systému při vysokém počtu připojených klientů. V rámci Azure SignalR free subscription bylo možné otestovat maximálně 20 připojených klientů najednou. I přes toto omezení byla aplikace schopna fungovat bez problémů, což naznačuje stabilitu a efektivitu implementace pro menší až středně velké projekty. Výsledky zátěžových testů poskytly cenné informace o výkonu systému a potvrdily, že při plánovaném nasazení bude aplikace spolehlivě reagovat na reálné uživatelské zátěže.

Závěr

V rámci této práce bylo úspěšně navrženo a implementováno robustní řešení pro interaktivní kvízovou aplikaci. Moderní .NET technologie, konkrétně Blazor, ASP.NET Core, SignalR a SQL Azure, umožnily vytvořit systém, který podporuje real-time komunikaci, bezpečný přístup a škálovatelnost. Funkční testy potvrdily správné fungování klíčových funkcí, zátěžové testy v rámci omezení Azure SignalR free subscription ukázaly stabilitu při připojení až 20 klientů a bezpečnostní testy ověřily efektivitu implementovaných autentizačních a autorizačních mechanismů. Celkově projekt demonstruje, že moderní .NET ekosystém je schopen nabídnout komplexní a efektivní řešení pro interaktivní vzdělávací aplikace, přičemž výsledná aplikace splňuje náročné požadavky na funkčnost, výkon i bezpečnost.

Seznam zkratek a odborných výrazů

.NET

.NET – platforma od Microsoftu pro vývoj aplikací, která umožňuje vytvářet robustní a škálovatelná řešení zejména v jazyce C#.

Blazor

Blazor – framework v rámci .NET ekosystému, který umožňuje vývoj interaktivních webových aplikací s využitím C# místo JavaScriptu.

SignalR

SignalR – knihovna pro real-time komunikaci mezi serverem a klientem, která zajišťuje okamžitou synchronizaci dat napříč připojenými uživateli.

JavaScript

JavaScript – skriptovací jazyk používaný pro dynamické a interaktivní chování webových stránek.

Azure

Azure – cloudová platforma od Microsoftu poskytující řadu služeb, včetně hostingu, databázových řešení a monitoringu.

JWT

JWT (JSON Web Token) – standard pro bezpečný přenos informací mezi stranami, využívaný především pro autentizaci a autorizaci uživatelů.

UI/UX

UI (User Interface) a UX (User Experience) – pojmy popisující design uživatelského rozhraní a celkový zážitek uživatele při interakci s aplikací.

HttpClient

HttpClient – třída v .NET, která umožňuje odesílání HTTP požadavků a komunikaci s REST API.

HMACSHA512

HMACSHA512 – kryptografický hashovací algoritmus používaný k bezpečnému hashování hesel a ověřování integrity dat.

Git

Git – distribuovaný systém pro správu verzí, který umožňuje efektivní sledování změn v kódu a spolupráci mezi vývojáři.

GitHub

GitHub – online platforma pro hostování repozitářů s využitím Git, podporující spolupráci a verzování kódu.

LINQ

LINQ (Language Integrated Query) – sada funkcí v .NET, která umožňuje psaní dotazů přímo v programovacím jazyce pro práci s kolekcemi dat.

CRUD

CRUD – zkratka pro Create, Read, Update, Delete; základní operace prováděné při práci s daty v databázích.

PIN

PIN (Personal Identification Number) – číselný kód používaný pro autentizaci či připojení uživatele, například při vstupu do herních lobby.

LocalStorage

LocalStorage – webové úložiště umožňující ukládání dat přímo v prohlížeči, což podporuje zachování dat mezi jednotlivými relacemi.

REST API

REST API (Representational State Transfer Application Programming Interface) – architektonický styl pro komunikaci mezi klientem a serverem prostřednictvím HTTP požadavků.

C#

C# – programovací jazyk vyvinutý Microsoftem, úzce spjatý s .NET, známý svou moderní syntaxí a využitím při tvorbě robustních aplikací.

Seznam obrázků

Databázový Diagram 1	11
----------------------------	----

Použité zdroje

1. **OpenAI**. chat.openai.com. *chat.openai.com*. [Online] 2025. <https://chat.openai.com>.
2. **Kahoot!** kahoot.com. *kahoot.com*. [Online] 2025. <https://kahoot.com/>.
3. **Microsoft**. Overview of ASP.NET Core SignalR. *Learn Microsoft*. [Online] 02. 12 2024. <https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction?view=aspnetcore-9.0>.
4. —. Dotnet Microsoft. *What is .NET?* [Online] 2025. <https://dotnet.microsoft.com/en-us/learn/dotnet/what-is-dotnet>.
5. —. Learn Microsoft. *Introduction to authorization in ASP.NET Core*. [Online] 12. 03 2024. <https://learn.microsoft.com/en-us/aspnet/core/security/authorization/introduction?view=aspnetcore-7.0>.
6. —. Learn Microsoft. *ASP.NET Core Blazor hosting models*. [Online] 11. 12 2024. <https://learn.microsoft.com/en-us/aspnet/core/blazor/hosting-models?view=aspnetcore-7.0>.
7. —. Learn Microsoft. *Entity Framework Core*. [Online] 11. 12 2024. <https://learn.microsoft.com/en-us/ef/core/>.
8. —. Learn Microsoft. *Architect Modern Web Applications with ASP.NET Core and Azure*. [Online] 18. 01 2025. <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/>.
9. —. Azure Microsoft. *Azure pricing*. [Online] 2025. <https://azure.microsoft.com/en-us/pricing/#Pricing-by-product>.
10. —. Learn Microsoft. *Azure SQL documentation*. [Online] 2025. <https://learn.microsoft.com/en-us/azure/azure-sql/?view=azuresql>.
11. **GitHub**. GitHub Docs. *About GitHub and Git*. [Online] 2025. <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>.
12. **Auth0**. JWT. *Introduction to JSON Web Tokens*. [Online] 2025. <https://jwt.io/introduction>.
13. **Microsoft**. Learn Microsoft. *ASP.NET Core Middleware*. [Online] 17. 06 2024. <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-7.0>.
14. **Bootstrap**. Getbootstrap. *Build fast, responsive sites with Bootstrap*. [Online] 2025. <https://getbootstrap.com/>.
15. **Microsoft**. Learn Microsoft. *ASP.NET Core Blazor*. [Online] 12. 12 2024. <https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-9.0>.
16. —. Learn Microsoft. *ASP.NET Core Blazor routing and navigation*. [Online] 12. 11 2024. <https://learn.microsoft.com/en-us/aspnet/core/blazor/fundamentals/routing?view=aspnetcore-7.0>.

17. **Stehlík, Michal.** *Návod k maturitním pracím 2020.* Liberec : Albatros, 2020.

A. Seznam příložených souborů

Na při

- **MP2025-Housa-Jan-P4A-Multiplatformní_kvízová_aplikace.docx** – editovatelná verze dokumentace maturitní práce
- **MP2025-Housa-Jan-P4A-Multiplatformní_kvízová_aplikace.pdf** – tisknutelná verze dokumentace maturitní práce
- **Aplikace** – zdrojové kódy

Soubory jsou k dispozici ve veřejném GitHub repozitáři, který naleznete prostřednictvím přiloženého QR kódu:

